scheduler —all automatically:

1. Connect to the Internet at the scheduled time
2. Download the file present at the provided URL
3. Log out from the Internet service at the scheduled time
4. Shut down our system at the scheduled time

Before we dive into the subject, there are two things to make a note of: make sure you log in to the system as the root user, and save all scripts and text files that we will create in a single directory.

Also, the first and third tasks listed above—connecting to the Internet and performing the logout—apply only for those connections that demand users to log in by submitting their user name and password into a Web page located at the ISP's server, and to log out by clicking on a logout link (for example, Sify). People with always-on connections (like Airtel, BSNL, etc) need not worry about these two tasks as most of these connections ensure connectivity as soon as the modem/router is connected to the system. Those belonging to this category only need to worry about downloading and shutting down the system at the right time.

We will run three independent scripts: **start.sh**, **logout.sh** and **shutdown.sh** — each from three different terminal windows. **start.sh** will control logging in to the Net and starting the download. **logout.sh** will log out the system, and **shutdown.sh** will shut down the system.

A note on *lynx-cur*

lynx-cur is a command-line browser—there's no mouse and no graphics here; you can only browse using keystrokes. If we are able to create a file that contains the sequence of key-strokes required to navigate a particular Web page, and if we are able to effectively transfer this data to the browser, then logging in and out are easy.

This is where **lynx-cur** comes to our aid. It has the striking feature of reading the key strokes from a file and then sequentially applying them on the given Web page, creating the same effect as if we were manually doing it. All you need to do is to provide an option: **-cmd_script=<file name>** while invoking the browser to browse a page. Lynx will read the key strokes from the specified file. For example:

```
lynx -cmd_script=<file name> <URL of the web page>
```

So the problem of an automatic login and logout is resolved provided we have a file that has the key-strokes data logged exactly in the same way the browser would like to read and interpret it.

But how do we create such a file? Lynx has an answer for this one too. Just like the **-cmd_script** option, Lynx provides another option called **-cmd_log**. With this option, the browser will log the key strokes we execute while browsing to a specified file (after it gets invoked and till we quit). For example:

```
lynx -cmd_log=<file name> <URL of web page>
```

Logging in

Command-line browsing in Lynx is all about moving the cursor in between hyperlinks and text fields (user name and password) in the login page by using the **Tab** key, and entering text into the text fields. In order to follow a link, we select the link by using **Tab** and press **Enter**. So it is understood that we will keep a text file created by Lynx, which it will use while running the **start.sh** script. To create the file, run the following command:

```
lynx -cmd_log=dir_in <URL of your login page>
```

The browser will open the login page. Now move the cursor to the text field where we are supposed to enter the user name and password, and enter the necessary data. After that, try to log in by moving to the login link and pressing **Enter**.

Remember that the process of logging in may vary with ISPs and the type of connection you have. For example, those with always-on (aided by a dedicated router/modem) connections can skip building the **start.sh** script, as they will get automatically logged in after they boot their computer. Here I am sticking to the core idea of a Web-based login method—i.e., reading keystrokes from a file, and even that, only if users have to submit their profile to authenticate their identity. Note that the name of the file given here is **dir_in** keeping in mind it contains directions for the browser to log in.

Now that you have logged in, you can close your browser. Press **q** and it will ask for confirmation. Enter **y**. That's it. Our **dir_in** file is ready.

You can see that **dir_in** has got all the keystrokes you've made. Also, you can see your user name and password. So, make sure you have changed the read, write and execution permissions of the file:

```
chmod 700 dir_in
```

Hereafter, if you need to log in, you can simply run the following command:

```
lynx -cmd_script=dir_in <URL of your login page>
```

...which will fill your authentication details, submit the data, and will finally quit from the browser to return to the terminal.

Finally, save the above command in a file named **login.sh**.

What if you're not able to connect the first time you invoked **login.sh**. In that case, you will have to run **login.sh** repeatedly till the connection is established. For that, embed the above command in the script within a **while** loop as follows:

```
#login.sh
#!bin/bash
```